

TD05 : Architecture MVC Avancée (1/2)

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	5

1) Problématique

Nous souhaitons apprendre à bien organiser notre code grâce au design pattern MVC (Modèle - Vue - Contrôleur). Pour cela nous allons effectuer plusieurs exercices afin de séparer notre site web en trois parties distinctes (le Modèle pour la gestion des données, la Vue pour l'affichage en HTML et le Contrôleur avec le Routeur pour gérer les actions) en s'aidant d'outils comme Xampp et Notepad++.

2) Exercices

Exercice 1 : Schéma MVC

On commence par se remémorer le schéma du MVC du TD précédent : l'utilisateur demande une action au routeur, le routeur appelle la bonne méthode du contrôleur, le contrôleur demande les données au modèle, puis le contrôleur transmet ces données à la vue qui se charge d'afficher le résultat en HTML.

Exercice 2 et 3 : Classe File et passage à index.php

1) Création de File.php dans le dossier lib/

Pour ne plus avoir de problème avec les chemins relatifs qui changent selon le dossier, on crée une classe File dans un nouveau dossier lib/. Sa fonction build_path utilise les constantes DIRECTORY_SEPARATOR et __DIR__ pour fabriquer des chemins absolus portables :

```
C:\xampp\htdocs\TD5\lib\File.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

1 <?php
2 class File {
3     public static function build_path($path_array) {
4         $DS = DIRECTORY_SEPARATOR;
5         $ROOT_FOLDER = __DIR__ . $DS . "..";
6         return $ROOT_FOLDER . $DS . join($DS, $path_array);
7     }
8 }
9
10
```

2) Création de index.php à la racine

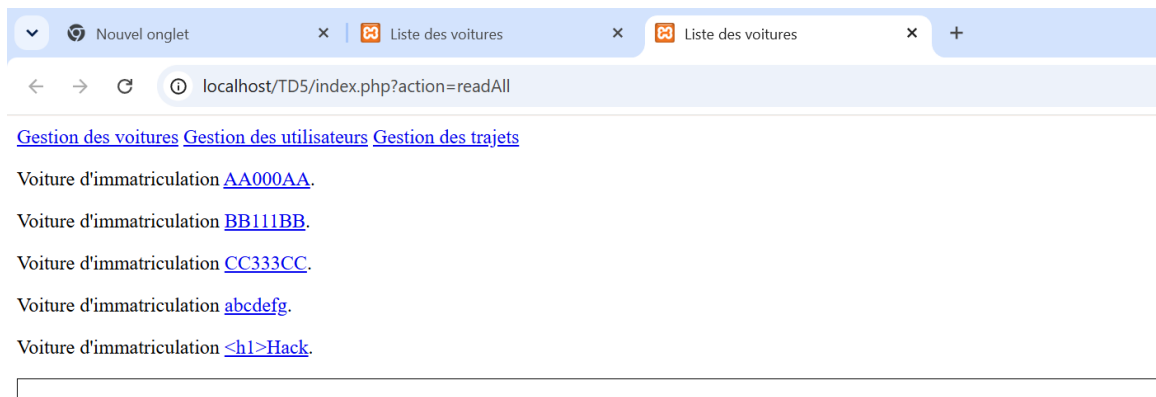
On crée le fichier index.php à la racine de notre site TD5. Ce fichier sert maintenant de page d'entrée unique : il charge File.php puis appelle le routeur avec build_path :

```
C:\xampp\htdocs\TD5\index.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

1 <?php
2 $DS = DIRECTORY_SEPARATOR;
3 $ROOT_FOLDER = __DIR__;
4 require_once __DIR__ . $DS . "lib" . $DS . "File.php";
5 require_once File::build_path(array("controller", "routeur.php"));
6
7
```

3) Test de la nouvelle page d'accueil dans le navigateur

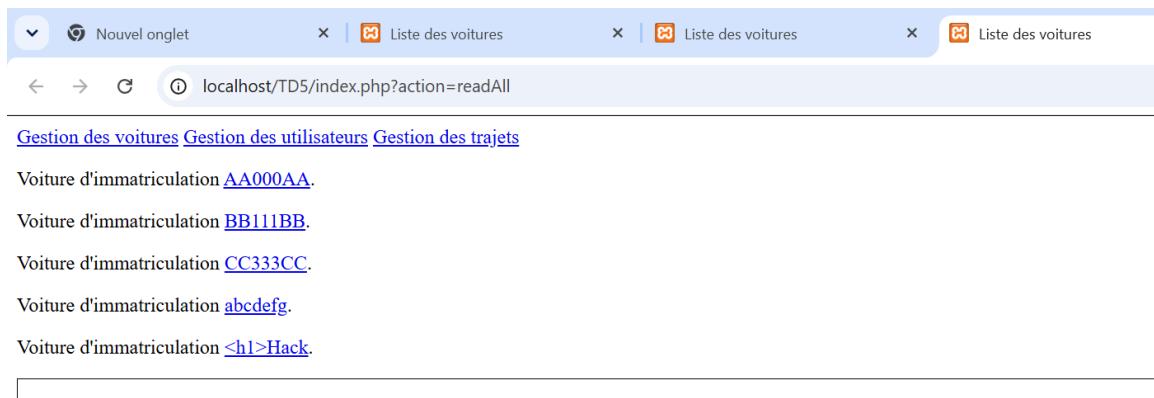
On teste en ouvrant l'adresse <http://localhost/TD5/index.php?action=readAll> :



Exercice 4 et 5 : Sécurité des vues (Échappement HTML et URLs)

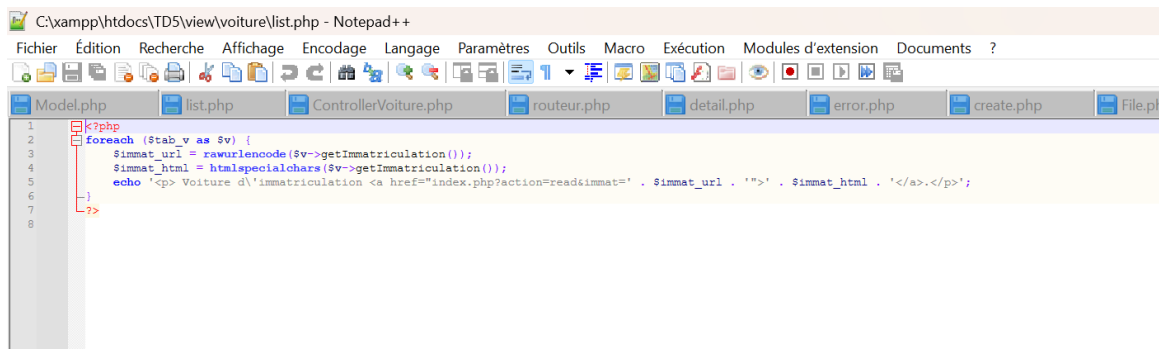
1) Test d'une injection HTML dans l'immatriculation

Si un utilisateur rentre `<h1>Hack` comme immatriculation, le navigateur interprète la balise HTML et l'affiche en très gros. Pour empêcher ça, on utilise la fonction `htmlspecialchars()` dans toutes nos vues pour que les balises HTML soient transformées en texte inoffensif :



2) Échappement des URLs avec rawurlencode dans list.php

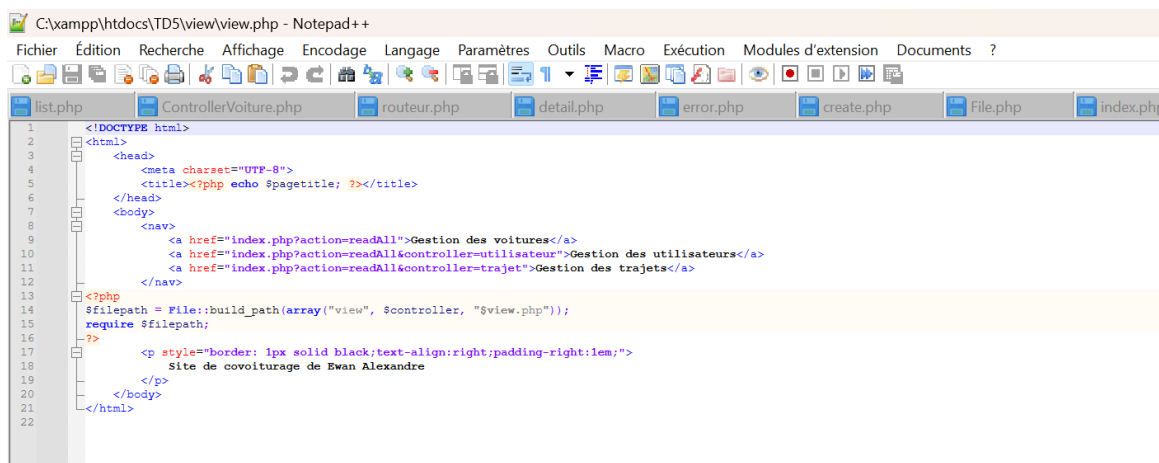
De la même manière, si une immatriculation contient des caractères réservés comme & ou =, le lien dans la liste peut être cassé. Dans list.php, on applique donc rawurlencode() pour la partie dans le lien, et htmlspecialchars() pour le texte affiché :



Exercice 6, 7 et 8 : Vues modulaires et vue générique view.php

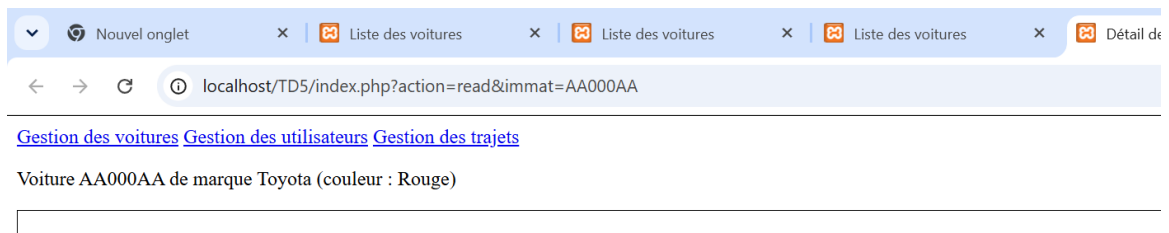
1) Création de la vue générique view.php

Pour ne pas répéter le code du début et de la fin de page sur chaque fichier, on crée view/view.php. Elle contient le haut de la page avec un menu de navigation, charge la vue demandée grâce à \$filepath, et affiche un pied de page en bas :



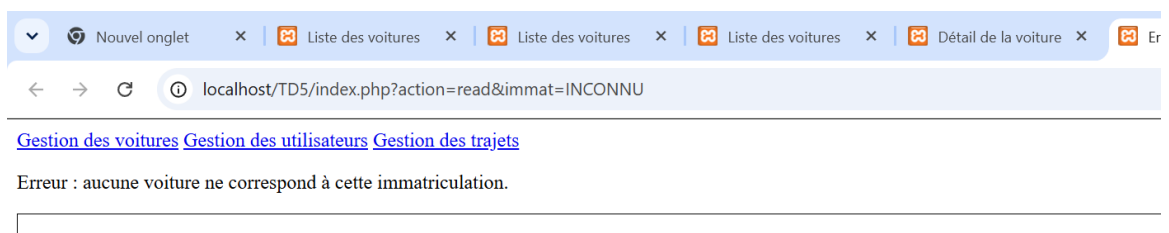
2) Test de la vue détail avec le menu et le footer

On teste la page de détail d'une voiture en allant sur <http://localhost/TD5/index.php?action=read&immat=AA000AA> :



3) Test de la vue d'erreur

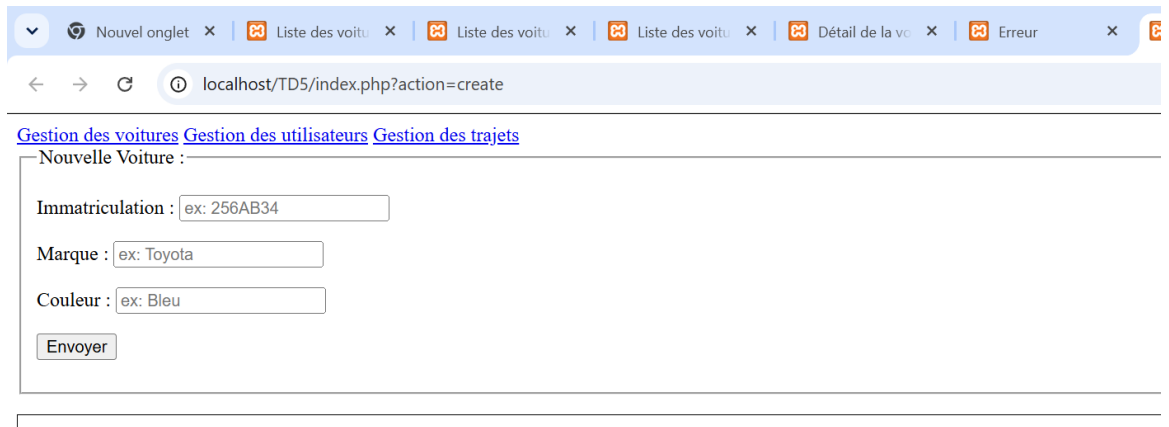
Si on demande une voiture qui n'existe pas en base, la vue d'erreur s'affiche avec le menu et le footer :



Exercice 9 : Concaténer des vues (Action created)

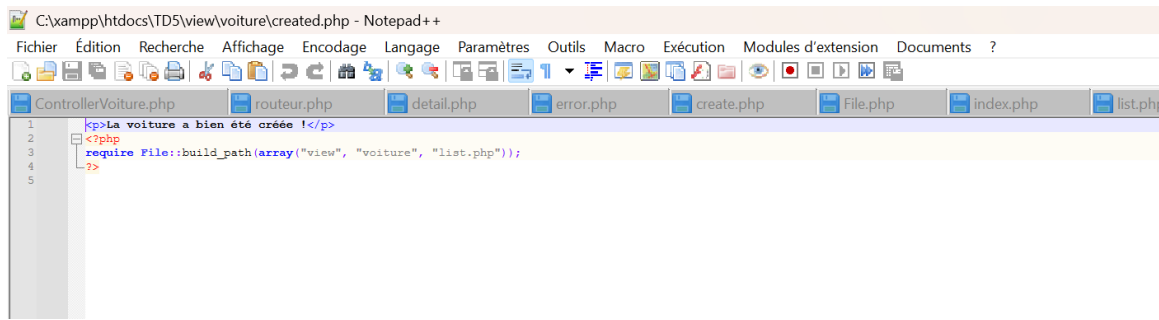
1) Vue du formulaire d'ajout create.php

On ouvre le formulaire d'ajout sur <http://localhost/TD5/index.php?action=create> :



2) Création de created.php et enchaînement avec la liste

On crée la vue created.php qui affiche le message de confirmation puis appelle immédiatement la vue list.php. Ainsi, dès qu'une voiture est créée, on voit le message de succès suivi directement de la liste actualisée :



```
C:\xampp\htdocs\TD5\view\voiture\created.php - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

1  <p>La voiture a bien été créée !</p>
2
3  <?php
4  require File::build_path(array("view", "voiture", "list.php"));
5  ?>
```

3) Conclusion

En conclusion, ce TD nous a permis d'améliorer notre architecture MVC pour la rendre plus professionnelle. Nous avons bien compris comment utiliser des chemins absolus portables avec la classe File et index.php, comment sécuriser l'affichage de nos données avec htmlspecialchars et rawurlencode, et comment modulariser nos vues grâce à une vue générique qui rassemble le menu, le contenu et le pied de page.